

1.1 Introduction

What is Python?

Python is a popular, high-level programming language created by **Guido van Rossum** and first released in **1991**. It emphasizes **readability** and uses an **indentation-based structure** to make code clean and easy to follow. Today, Python is widely used for a large variety of applications.

Benefits of Python

- **Easy to Learn:** Its syntax almost resembles plain English.
- **Versatile:** Works for console, web development, data analysis, automation, AI, and more.
- **Large Community:** Extensive libraries and frameworks are available for almost any task.
- **Cross-Platform:** Python works on Windows, macOS, Linux, and more.

Drawbacks of Python

- **Speed:** Python is slower and less efficient compared to languages like C++ because it's interpreted.
- **Runtime Errors:** Python is dynamically typed, leading to potential runtime errors. Because of this, it can be hard to find/prevent errors properly in advance, especially in large projects.

Compiled Vs Interpreted Languages

- **Compiled Languages:** Code is translated into machine code before execution (e.g., C, C++).
- **Interpreted Languages:** Code is executed line-by-line by an interpreter (e.g., Python, JavaScript).

Usecases/Applications

Python is used in a wide range of applications:

- **Web Development:** Popular frameworks like Django and Flask make web development faster and easier.
- **Data Science & Analytics:** Libraries such as pandas, NumPy, and Matplotlib are widely used for data manipulation and visualization.
- **Machine Learning & AI:** Python's simplicity makes it a go-to language for machine learning and artificial intelligence applications, with libraries like TensorFlow and scikit-learn.
- **Automation:** Python is commonly used to write scripts for automating repetitive tasks, such as file management and web scraping.

Syntax

Python's syntax is minimal and highly readable. In general Python code will run line by line from top to bottom, more complex behaviour is possible using [Control Flow](#).

- `print()` is used for [output](#), `input()` for [input](#), and `#` for line [comments](#)
- Python supports all basic (mathematical) [Operators](#) and many more
- [Variables](#) automatically get a [datatype](#) that can change however you like
- Python is **case sensitive**, this means that if only upper/lower case of characters is different, it is a completely different thing, for example `test` is not the same as `Test`
- **Indentation:** Python relies on indentation to define code blocks, unlike many other languages that use braces `{}`. Proper indentation is required, or the code will throw an error. A colon `:` is used to indicate the start of a new block, e.g. `if` statements, `loops`, `functions`, ...

```
1  # A simple comment
2  print("Hello, World!") # This prints a message to the console
3
4  # Variables don't need explicit types
5  x = 10 # Integer
6  y = 3.14 # Float
7  name = "Python" # String
8
9  # Indentation is used for blocks
10 if x > 5:
11     print("x is greater than 5")
```

Run Python Online

There are websites that easily allow you to run Python code online. If you want to run Python online you have to be careful that what you want to do works and behaves similar to actually running the code on your machine. Many online compilers struggle with error messages, debugging, input, imports, multiple files, (web-)applications,

Below you can find a few websites that overall do a good job. It can be very helpful to have one of them open on the side while reading this tutorial, so that you can always copy, paste, run, edit, ... code from the notes to understand it better.

Code Chef

<https://www.codechef.com/blogs/how-to-use-code-visualizer>

This website has the advantage of also including a visualizer. This enables you to run code line by line, see variables and control flow and understand it much easier.

[Code Chef Visualizer Tutorial](#)

The drawback of this is that it will run the code on a separate machine so you have to provide all input values before running

Online GDB

https://www.onlinegdb.com/online_python_compiler

This website has the advantage of being very similar to a real command line execution. Inputs are in real time and just like in the command line. There is also Debugger available in case you want/have to use it.

[Online GDB Debugger Tutorial](#)

Google Colab

<https://colab.research.google.com/>

Google Colab is a powerful online tool for running Jupyter Notebooks^[1] online. It's a great fit for bigger projects and cloud usage of Jupyter. It also supports collaboration and GPUs for use cases like machine learning.

Python Installation

Even though there are good online tools, running Python on your own machine still has many benefits. To install Python, go to the [official website](#), download, execute and follow the instructions. Please don't forget to click the checkbox "Add to PATH"^[2] at the very start of the installation.

Visual Studio Code Installation

In theory you can use any text editor to write your code. In practice, it is much easier and better to use a specialized Code Editor usually called IDE (Integrated Development Environment). They have a lot of additional features like syntax highlighting, auto suggestions and completions, documentations, debugging, combining multiple parts of the coding workflow, search and replace, keyboard combinations, version management, ... There are a lot of options, also depending on coding language and application. In my opinion the best and most versatile option is Visual Studio code.

To install it just go to the [official website](#), download, execute and follow the instructions. To understand VSCode you can read the [documentation](#) or watch one of the many videos like [this one](#) or [this one](#). To properly use Python you will need to install the "Python" Extensions, which you can do in the VSCode Extensions tab.

Understanding and using your IDE well can make your life easier, enable new potential and make you much faster when writing code.

Questions

- ☐ What is programming language?
- ☐ What is Python?
- ☐ What are the benefits of Python?
- ☐ What are the drawbacks of Python?
- ☐ Explain compiled vs interpreted languages in your own words. What type is Python?
- ☐ What are the use cases of Python?
- ☐ What are general principals of the Python syntax.

-
1. Project Jupyter is a combination of Python code and formatted Text for Documentation, it's great for explaining code, visualizations, ... ↵
 2. PATH is basically a system internal variable that stores all things that can be accessed by default when running code from the command line. If you don't add Python to PATH you might not be able to run it properly or always have to tell exactly where your Python installation is. ↵

1.2 Output

Summary

- `print()` : Outputs values of any datatype to the console.
- **Defaults:** Adds a line break; blank lines if no arguments are given.
- **Separator** (`sep`) for value separation.
- **End** (`end`) for what follows the output.

Simple Print

Python has one primary function for console output called `print()` . In general you can give the print function any value of any [datatype](#) and it will print it to the console.

Print Hello World

This for example will print the message "Hello World" to the console.

```
1 print("Hello World") # -> Hello World
```

Printing Multiple Datatypes

You can also print any other values of any [datatype](#) using this function.

```
1 print(12)           # -> 12
2 print(2.5)          # -> 2.5
3 print(True)         # -> True
4 print("Some more text") # -> Some more text
```

By default, after every printed value, there will be a [line break](#), so the next value will be in a new line. The code above will output the following text to the console. You can see that each output is in its own line.

```
1 12
2 2.5
3 True
4 Some more text
```

Print without Any Arguments

If call the print function without any arguments, it will not print anything, but still make a new line.

```
1 print(1) # -> 1
2 print()
3 print(2) # -> 2
4 print()
5 print()
6 print(3) # -> 3
```

Bellow you see the exact output of these print statements.

```
1 1
2
3 2
4
5
6 3
```

Print Multiple Values

🕒 [less exam relevant](#) >

Less exam relevant, but still good/helpful to know.

The print function can take any number of comma separated values and print all of them. In the output, the values will be separated with spaces.

```
1 print("Hello", "World", "Python") # -> Hello World Python
```

Additional Arguments for Print

🕒 [less exam relevant](#) >

Less exam relevant, but still good/helpful to know.

The print function can not only take the values you want to print, you can also give some other arguments for special output.

Print Separator

You can tell the print function which value to put between the values you are printing using the `sep` parameter. You can give it any string to put between multiple values within the print, the default is a space.

```
1 print("Hello", "World", "Python", sep=" | ") # -> Hello | World | Python
2 print("Shopping List: Apple", "Banana", "Bread", "Milk", sep=", ") # -> Shopping List: Apple, Banana, Bread, Milk
```

Print End

You can tell the print function which value to put after all of the printed values using the `end` parameter. You can give any string, default is a new line (`\n`).

```
1 print("This doesn't end with a newline", end=" ---> ")
2 print("This continues", end=" ")
3 print("on the same line.")
```

```
1 This doesn't end with a newline ---> This continues on the same line.
```

Questions

- ☐ Explain `print()` in your own words.
- ☐ What will be the output of the following code?

```
1 print("Python", "is", "fun", sep="-", end="!")
2 print("Let's code.")
```

- ☐ What does the `sep` parameter do in the `print()` function? Provide an example.
- ☐ If the `print()` function is called without any arguments, what will be its behavior? Provide an example output.
- ☐ How can you ensure the output of two `print()` statements appears on the same line?

1.3 Input

Summary

- `input()` : Waits for user input and always returns a string.
- Saving Input:** Store the returned value in a variable for later use.
- Prompting:** Pass a string to `input()` to display a prompt before input.

Simple Input

Python has one primary function for console input called `input()`. It will wait for the user to input something on the console. As soon as the user presses `Enter` the function will return the value entered by the user and continue with the program.

Input will **always** return the value as a `string`. So if you want the input to be a different `datatype`, you have to explicitly convert it, but you have to be careful. Taking input of some kind that has an unexpected datatype or value is one of the most common error cases. If you take input you should tell the user what you expect, check if the entered value is valid, and include exception handling.

```
1 input()
```

You can save the value returned by `input` in a `variable` to use it later/multiple time.

```
1 x = input()
2 print("The user entered the value:", x) # -> The user has entered the value {?}
```

You can also use the value returned by `input()` directly.

```
1 print("The user entered the value:", input()) # -> The user has entered the value {?}
```

Input with Prompt

If you want, you can give the `input` function one argument. It will output this value directly before taking input. This is usually used to ask and inform the user what input is expected

```
1 x = input("Please input a number: ") # -> Please input a number:
2 print("The user entered the number:", x) # -> The user has entered the number {?}
```

Questions

- ☐ Explain `input()` in your own words.
- ☐ What does the `input()` function always return, regardless of the type of input?
- ☐ How can you store the value returned by `input()` for later use?
- ☐ What happens when you call `input()` with no prompt message?
- ☐ What does providing an argument to `input()` do, and how is it typically used?
- ☐ How would you modify the code to prompt the user for their name and greet them with a personalized message?

```
1 x = input()
2 print("Hello, ", x)
```

1.4 Comments

Comments are for the Programmers and are completely ignored by the computer. They are usually used to document and explain code.

However, writing one irrelevant comment per line is as bad as having no documentation/comments at all, so you have to find a balance. Explain important logical steps, code blocks and decisions. Do not only write comments if you need them right now, also write comments if someone else needs them, or you need them when looking at code a year later.

Singleline Comments

Everything in a line after a hash `#` is part of the single line comment and will be ignored by the program.

```
1 # This is some example code
2 print("Hello") # this line will print the message "hello"
3 # print(1)      this will not do anything because the print is after the comment
```

```
4 print(1) # new line so print is working again and printing 1
```

```
1 Hello
2 1
```

Docstrings/Multiline Comments

If you want to have a bigger commented documentation and don't want to put a `#` in front of every single line, you can use a docstring. Everything between three double quotes `"""` is interpreted as a docstring, so basically a multiline comment.

```
1 print(1)
2 """Here is the start
3 this is still part of the comment
4 print(2) is still in the comment"""
5 print(3)
```

```
1 1
2 3
```

Documenting Functions/Methods

🕒 less exam relevant >

Less exam relevant, but still good/helpful to know.

Docstrings are often used to document the definition of a function, method or class. You can put an explanation of what the class is doing, what arguments it can take, ... directly under the define/class keyword. Most IDEs will even display this explanation to the programmer when using the function/method/class.

```
1 def hello(name):
2     """
3     Prints a greeting message with the given name.
4
5     Args:
6         name (str): The name to include in the greeting.
7     Returns:
8         None
9     """
10    print(f"Hello {name}")
11
12    hello("Andreas")
```

[Open: FunctionDocstringExample.jpg](#)



Questions

- ☐ Explain comments in your own words?
- ☐ What is the purpose of comments?
- ☐ When should you write a comment?
- ☐ Explain single line comments.
- ☐ Explain docstrings/multiline comments.

1.5 Datatypes

A datatype defines how different types of information/data are stored and behave. More details regarding the behaviour can be found on the pages [Operators](#), built in methods and functions, ...

For almost every datatype in Python, there is a function with the same (or similar) name that is used to create an object of that datatype or convert something from a different datatype.

Name	Function	Example	Explanation
Integer	<code>int()</code>	<code>0</code> , <code>11</code> , <code>-3</code>	whole number

Name	Function	Example	Explanation
Float	float()	1.5 , 1e-4	rational/floating point number
Boolean	bool()	True , False	truth value, can only be True(1) or False(0)
String	str()	'abc' , f"x:\n{x}"	text, can include, numbers, special characters, ...
List	list()	[1, 1, 2, 3]	mutable collection of any datatype
Tuple	tuple()	(1, 1, 2, 3)	immutable collection of any datatype
Set	set()	{2, 1, 3}	mutable collection of any datatype without order, indexing or duplicates
Dictionary	dict()	{"a": 1, "c": "5"}	mutable mapping of key and value pairs of any datatype
None		None	None means that there is no value
Complex	complex()	1j , 1.2-0.5j	complex number with real and imaginary part
Range	range()	range(8) , range(2, 99, 3)	range of numbers
Generator			iterator over a dataset, created by the programmer

🕒 not exam relevant >

These functions/methods are mostly not exam relevant.

There are a few built in functions/methods to check if something is a specific type of data. These are helpful to check input, potentially before converting it.

- **String Methods:** `.isdigit()` , `.isalpha()` , `.isalnum()` , `.isdecimal()` , `.isnumeric()` , `.isspace()` .
- **Type Checking:** `isinstance(obj, type)` , `type(obj)` , `callable(obj)` .
- **Regular Expressions**

Basic Datatypes

Integer

The datatype Integer can hold any whole number including negatives and zero. You can use the function `int()` to convert a value to an Integer, or create one with value `0` if no argument is given.

```
1 1 4 -2 0 -4 962 -123456789
```

Float

The datatype Float can hold any rational/floating point number including negatives and zero. You can use the function `float()` to convert a value to a Float, or create one with value `0.0` if no argument is given.

```
1 1.5 -0.5 0.0 3.3333 1e-4 2.5e3 -5e6
```

Scientific Number Notation

🕒 not exam relevant >

This topic is not exam relevant.

Python supports the scientific notation of writing numbers as a power of 10 for integers and floats. To do this you can give numbers in the format " number e power ". So for example `1e3 == 1 * 10**3 == 1000` or `-1.5e-2 == -1.5 * 10**-2 == -0.015` .

Boolean

The datatype Boolean can hold a truth value. There are only two possible options for this value True or False (can also be thought of as 1/0 Yes/No On/Off). This datatype is very important for Control Flow like Conditions, because it is usually based on many "Yes/No Questions", which are answered with a Boolean value. You can use the function `bool()` to convert a value to a Boolean, or create one with value `False` if no argument is given.

Collection Datatypes

General

Bellow you find a table of some key properties of the collection datatypes available in Python.

Name	Mutable	Iterable	Indexing	Slicing	Common Use Case
String	No	Yes	Yes	Yes	Text processing and manipulation
List	Yes	Yes	Yes	Yes	Storing and manipulating mutable sequences
Tuple	No	Yes	Yes	Yes	Fixed-size collections of items
Set	Yes	Yes (but no order)	No	No	Ensuring unique elements in a collection
Dictionary	Yes	Yes (keys, values, or both)	Keys	No	Key-value pair storage and lookups

Name	Mutable	Iterable	Indexing	Slicing	Common Use Case
Range	No	Yes	Yes	Yes	Generating sequences of numbers
Generator	No	Yes	No	No	Efficient iteration over large datasets

Mutable

A mutable object can be modified after it is created. This means you can change, add, or remove parts or all of its content without creating a new object. For example, you can change the value of the second item in a list, but you can not change the value of the second item in a tuple or string. All basic datatypes are actually immutable. You can find the difference in behaviour between a mutable and immutable variable [here](#).

Iterable

An iterable is any object that can return its elements one at a time, allowing it to be used in a loop (e.g., `for` loops).

Indexing

Indexing refers to accessing individual elements of a sequence using their position (index). Python uses zero-based indexing, meaning the first element has an index of `0`, second one has an index of `1` ... You can also start indexing from the end of the sequence using negative numbers starting from `-1`, so the last element has an index of `-1`, the second last has an index of `-2` ...

A [dictionary](#) does not have classic ordered indexing with numbers. Instead, each element is a combination of key and value, and you use the key instead of an index

```
1  [3, 1, 7, 4, 5, 2] # list of values
2  0 1 2 3 4 5 # positive indexes
3  -6 -5 -4 -3 -2 -1 # negative indexes
```

Indexing can be used to get the value of a element or also to set/change it. To use it you first put the sequence and then the index inside square brackets `sequence[index]`.

```
1  listName = [3, 1, 7, 4, 5, 2] # create list and save in variable listName
2  print(listName[2]) # get and print the third element of the list -> 7
3  listName[0] = -1 # set the first element of the list to -1
4  print(listName) # print the whole list -> [-1, 1, 7, 4, 5, 2]
```

Slicing

 **partly exam relevant** >

Basics are relevant, more advanced usage is not necessary.

Slicing is used to select a subsequence/portion of a sequence, instead of only on element. You can specifying a start, stop, and step.

```
1  sequence[start:stop:step]
```

I you leave one of these blank, it will have a default value

- `start` : The starting index (**inclusive**). Defaults to `0` (first element) if omitted. Negative values work the same way as with indexing.
- `stop` : The ending index (**exclusive**). Because it is exclusive, the element with this index is not included, to include the last element you have to use one more then its index or `-1`. Defaults to `-1` (last element) if omitted. Negative values work the same way as with indexing.
- `step` : The interval between elements. Defaults to `1` (every element without skipping). Negative values reverse the order of the sequence.

If you get elements of a sequence using slicing, it will create a shallow copy of the sequence and give it to you.

```
1  listName = [3, 1, 7, 4, 5, 2, 9] # create list and save in variable listName
2  # Index      0 1 2 3 4 5 6
3  print(listName[1:3])           # elements from the 2nd (inclusive) to the 4th (exclusive) -> [1, 7]
4  print(listName[:4])            # elements from the 1st to the 5th -> [3, 1, 7, 4]
5  print(listName[2:])            # elements from the 3rd to the last (inclusive) -> [7, 4, 5, 2, 9]
6  print(listName[:])            # all elements -> [3, 1, 7, 4, 5, 2, 9]
7  print(listName[0:5:2])         # every second element from the 1st to the 6th -> [3, 7, 5]
8  print(listName[:3])           # every third element from the whole sequence -> [3, 4, 9]
9  print(listName[-5:-1:1])       # elements from the 5th last (inclusive) to the last (exclusive) -> [7, 4, 5, 2]
10 print(listName[::-1])          # all elements in reverse order -> [9, 2, 5, 4, 7, 1, 3]
11 print(listName[6:1:-2])        # every second element from the 7th to the 2nd in reverse order -> [9, 5, 7]
```

You can also set multiple values at the same time using slicing. If you don't give the correct numbers of values to set/replace, you might get an error.

 **not exam relevant** >

Not exam relevant, rarely used, but sometimes really helpful.

```
1  listName = [3, 1, 7, 4, 5, 2, 9] # create list and save in variable listName
2  listName[1:4] = [10, 20, 30] # replace elements from the 2nd to the 5th -> [3, 10, 20, 30, 5, 2, 9]
3  listName[-2:] = [400, 500] # replace elements from the 2nd last to the last -> [3, 10, 20, 30, 5, 400, 500]
4  listName[::2] = [1, 2, 3, 4] # replace every second element -> [1, 10, 2, 30, 3, 400, 4]
```

```
5  listName[:0] = [-1, -2] # insert two new elements at the start of the sequence -> [-1, -2, 1, 10, 2, 30, 3, 400, 4]
6  listName[2:5] = [] # remove the elements from the 3rd to the 6th -> [-1, -2, 30, 3, 400, 4]
```

String

The datatype String can hold any text including numbers and [special characters](#). In Python, all strings have to be within quotes single 'string' or double "string" quotes, using [tripple quotes](#) has a special behaviour. You can use the function `str()` to convert a value to a String, or create one with value "" if no argument is given. Strings also supports [indexing](#), [slicing](#) and some list functions/methods. But it is impossible to change individual elements/characters of a string, because it is not [mutable](#).

```
1  "text" 'string' "some long message" "line 1\nline 2" """tripple quotes string"""
```

You might think of string as a base datatype, but it actually behaves very similar to a [list](#) (or more precisely a [tuple](#)) of individual characters. So especially in low level languages you will find `character` as a base datatype and the string is just a collection of characters.

Special Characters

🕒 partly exam relevant >

Basics are relevant, more advanced usage is not necessary. First 5 rows of the table are relevant, the others not.

Python strings support many special characters. To use a special character first write a backslash \ to escape the string and tell Python that the next thing is a special character. After the backslash you can put one of the supported special characters.

Character	Description	Example
\n	Newline character. Moves the cursor to the next line.	"Hello\nWorld" → Hello World
\t	Horizontal tab. Adds a tab space.	"Hello\tWorld" → Hello World
\\	Backslash. Escapes the backslash itself.	"Path: C:\\Users" → Path: C:\Users
' or ''	Escapes a single quote within single-quoted strings.	'It\'s a test' → It's a test
" or ""	Escapes a double quote within double-quoted strings.	"She said \"Hello\"" → She said "Hello"
\r	Carriage return. Moves the cursor to the beginning of the line and overwrites characters.	"123456789\rWorld" → World6789
\b	Backspace. Removes the character before it.	"Hello\bWorld" → HellWorld
\x**	Character with a hex value (2 digits).	"\x48" → H
\u****	Unicode character with a 4-digit hex code.	"\u2764" → ❤️
\U*****	Unicode character with an 8-digit hex code.	"\U0001F600" → 😊

If you want to prevent special characters from being interpreted you can use raw strings `r"string"`. For example if you do `r"line1\nstill line1"` there will not be any newline.

Formatted String

Formatted strings in Python provide an easy way to insert values into a string. To make a formatted string put an `f` in front of the the starting string quotes. Inside a formatted string, all the values within curly brackets `{value}` will be inserted into the string. The datatype will automatically be converted and there is also support for operators, functions and special formatting. If you want to print a curly bracket within a f-string, you have to put a double curly bracket `{{}`.

```
1  b = 5
2  pi = 3.14159
3  name = "Andreas"
4  print(f"My name is {name} and I am {19} years old.") # -> My name is Andreas and I am 19 years old.
5  print(f"The sum of {a} and {b} is {a+b}")           # -> The sum of 1 and 5 is 6
6  print(f"The user inputted: {input()}")             # -> The user inputted: {?}
7  print(f"Pi with two decimal places: {pi:.2f}")     # -> Pi with two decimal places: 3.14
8  print(f"Documenting operations: {1 + 3 * 2 =}")    # -> Documenting operations: 1 + 3 * 2 = 7
```

Multiple Line String

🕒 less exam relevant >

Less exam relevant, but still good/helpful to know.

Instead of using single or double quotes at start end end of a string, you can also use them three times (`'''` or `"""`) to create a multiline string. So within tripple quotes you can have multiple lines of text and it will keep the format, linebreaks and spacing. You can also combine this with [Formatted String](#).

```
1  print("""This is a
2  multiline string
3      indentation is also
4      as in the code""")
5  print(f"""Below you can see an operation
```

```
6 {1 + 1 = }""")
```

```
1 This is a
2 multiline string
3     indentation is also
4     as in the code
5 Below you can see an operation
6     1 + 1 = 2
```

List

The datatype List can hold a sequence of any datatypes, it is even possible to put collection datatypes inside a list. All list elements have to be within square brackets `[]` and separated by commas `,`. You can use the function `list()` to convert an [Iterable](#) value to a list, or create an empty list `[]` if no argument are given. Strings also supports [indexing](#), [slicing](#) and many built in functions/methods. You can add, change or remove elements, because it is [mutable](#).

```
1 [1, 1, 2, 3] [] [1, "hello", 3.14, True] [[[1,2,3], [-1,-2,-3]],[[10,20], [-10,-20]]]
```

Tuple

You can think of a tuple like a [List](#) where it is impossible to add, change or remove elements. The reason for this is, that a tuple is not [mutable](#). All tuple elements have to be within parentheses/round brackets `()` and separated by commas `,`. You can use the function `tuple()` to convert an [Iterable](#) value to a tuple, or create an empty tuple `()` if no argument are given.

```
1 (1, 1, 2, 3) () (1, "hello", 3.14, True) ([[1, 2, 3], [-1, -2, -3]], [[10, 20], [-10, -20]])
```

Set

🕒 not exam relevant >

This topic is not exam relevant, but it is still good to know that this datatype exist and its ideas/behaviour.

A set is similar to a [List](#) but the elements are stored in a specific ordered. Because of this, [Indexing](#) is not possible. [Iterating](#) is possible, but there is no order defined for the elements. All set elements have to be within curly brackets `{}` and separated by commas `,`. You can use the function `set()` to convert an [Iterable](#) value to a set or create an empty set if no argument are given. You only create an empty set using the function, because [Dictionary](#) does also use curly brackets. You can add or remove elements, because it is [mutable](#). A set can only store not [mutable](#) elements. A set does not allow for any duplicate elements, if you have duplicates only one of them will be stored. Only a few of the list operators, functions and methods will work, but there are also some special ones for sets.

```
1 {1, 2, 3} set({1, 3.14, 'hello'}) {'hello', True, 2, 3.14}
2 print({1, 2, 1, 1, 3, 2, 3, 4, 5}) # duplicates removed, no specific order -> {3, 5, 1, 4, 2}
```

Dictionary

The datatype Dictionary creates a mapping between keys and a values. A value can be any datatype. A key has to be unique (every key only exists) and not [mutable](#). All dictionary elements have to be within curly brackets `{}` and separated by commas `,`. One element is a combination of a key and a value separated by a colon `:` like this `key:value`.

```
1 {"a": 1, "c": "5"} {} {1: (1,2,3), "a": 1, (1,2): 1}
```

You can use the function `dict()` to convert values to a dictionary, or create an empty dictionary `{}` if no argument are given. You can use the function `dict()` by giving no arguments, an [Iterable](#) of key and value pairs or keyword arguments.

```
1 print(dict()) # empty dictionary -> {}
2 print(dict([('name', 'Tom'), ('age', 25), ('city', 'Linz')])) # dictionary from pairs -> {'name': 'Tom', 'age': 25, 'city': 'Linz'}
3 print(dict(name='John', age=30, country='USA')) # dictionary from keyword arguments -> {'name': 'John', 'age': 30, 'country': 'USA'}
```

You can get the value of a key the same way you would use normal [indexing](#). You can add, change or remove elements, because it is [mutable](#). If you want to add a value you can use assignment with the new key with the syntax `dictName[newKey] = newValue`, if the key already exists it will change the value of the existing key.

```
1 person = {'name': 'Tom', 'age': 25, 'city': 'Linz'}
2 print(person["age"]) # print age of person -> 25
3 person["country"] = "Austria" # add new element with key "country" and value "Austria"
4 person["age"] = 19 # change the value of age to 19
5 print(person) # -> {'name': 'Tom', 'age': 19, 'city': 'Linz', 'country': 'Austria'}
```

Special Datatypes

None

The datatype None represents the absence of a value or a "null" value. It is often used to indicate that a variable or function does not have any (meaningful) value. None can be used as a placeholder, for example when defining a variable before its actual value is known. Having `None` is not an error it is just saying that there is no value/the value is nothing.

For example, if you try to get a variable value or list element that does not exist, you will get a error message. But storing/printing the value returned from a function that returns nothing is not an error, it just has the value `None`.

Complex

🕒 less exam relevant >

Less exam relevant, rarely used.

The datatype Complex represents a complex numbers. It has a real and an imaginary part, represented as $a + bj$, where a is the real part and b is the imaginary part. If you have a number with an imaginary part in Python, it will treat/save it as a complex number. You can use the function `complex()` to convert a value to a Complex, or create one with value `0j` if no argument is given. This function takes two numbers, the real part a and the complex part b , as arguments `complex(a, b)`.

```
1 1+3j 1j -3+1j 0.1-1.5j 1-20000j 1-2e4j -0.001-1j -1e-3-1j
```

You can access the real or imaginary part of a complex number individually using the `.real` or `.imag` attributes.

```
1 z = 3 + 4j
2 print(z.real) # -> 3.0
3 print(z.imag) # -> 4.0
```

Range

🕒 partly exam relevant >

Only understand that this is different from list, because it does not store all elements in memory and instead generates them on demand. More advanced usage is not necessary.

The datatype Range is created with the range function and generates a sequence of numbers. It might seem very similar to a list, but the key difference is that not all elements are saved in memory, instead they are generated only when they are actually used. It is commonly used in loops and is a memory-efficient [Iterable](#). Interestingly you can even [index](#) and [slice](#) range objects and will either a number or a new slice as a result. You can not add, remove or change elements of a range, because it is not [mutable](#).

Generator

🕒 not exam relevant >

This topic is not exam relevant. It is a bit advanced and rarely used.

The datatype Generator is very similar to [Range](#) in supporting memory efficient one by one [iterating](#) of elements. The key difference is that it supports all datatypes and you can define your own sequence generators. But they do not support [indexing](#) or [slicing](#) and it is not [mutable](#).

Bytes

Object Oriented Programming

🕒 only SDML and AECD >

This topic is an advanced topic that is only relevant for the SDM and AECD department. There will be a note only for this at the end.

You can basically define and use your very own datatypes using OOP or import special datatypes created by others. First you have to define it with all its values and behaviour in a class, then you can create objects based on this class.

Internal Details

Truthy and Falsy

In python any value can be evaluated (e.g. in a [logical operator](#), [condition](#), ...) like a boolean. Python will treat it as truthy (`True`) or falsy (`False`) based on its type and value.

- **Falsy Values:** `False`, `0`, `0.0`, `0j`, `""`, `[]`, `()`, `set()`, `{}`, `None`, `range(0)`
- **Truthy Values:** Everything else so not empty collections and not zero numeric values

```
1 print(bool(0-1j)) # True
2 print(bool([])) # False
3 print(bool(range(0))) # False
4 print(bool(0.000001)) # True
5 print(bool("")) # False
6 print(bool(" ")) # True
```

🔗 Additional not exam relevant topics for later >

Contents

Converting between Datatypes

Questions

- ☐ What is a datatype and what is its purpose?
- ☐ Explain Integer in your own words.
 - ☐ How do you define an Integer in Python?
- ☐ Explain Float in your own words.
 - ☐ What is the difference between an Integer and a Float?
 - ☐ Explain the scientific notation in python.
- ☐ 3 Explain Boolean in your own words.
 - ☐ What is the Boolean type in Python used for?
- ☐ Explain String in your own words.
 - ☐ Explain special characters in your own words.
 - ☐ Explain Formatted String in your own words.
 - ☐ Explain Multiline String in your own words.
- ☐ Explain List in your own words.
 - ☐ What is the difference between a List and a Tuple in Python?
 - ☐ Can Lists store different data types in Python?
 - ☐ Explain Mutable in your own words.
 - ☐ Explain Iterable in your own words.
 - ☐ Explain Indexing and Slicing in your own words.
 - ☐ Give an example of list inside a list and how you can index the inner list.
- ☐ Explain Tuple in your own words.
 - ☐ What makes a Tuple different from a List in Python?
 - ☐ Can you modify the items in a Tuple?
 - ☐ What are some use cases where you would prefer a Tuple over a List?
- ☐ Explain Set in your own words.
 - ☐ What are the main characteristics of a Set in Python?
 - ☐ How does a Set differ from a List or Tuple?
 - ☐ What values can you store in s set and what is special about this?
- ☐ Explain Dictionary in your own words.
 - ☐ How can you access values in a Dictionary?
 - ☐ What datatypes can you use a Dictionary key?
 - ☐ What datatypes can you use a Dictionary value?
- ☐ Explain None in your own words.
 - ☐ Give examples where you will get or use a None value.
- ☐ Explain Complex in your own words.
 - ☐ What is a Complex number in Mathematics and Python?
 - ☐ How can you get the real and imaginary part individually?
- ☐ Explain Range in your own words.
 - ☐ How does a Range differ from other types like a List?
- ☐ Explain Generator in your own words.
 - ☐ How does a Generator differ from other types like a List?
- ☐ Explain truthy and falsy in your own words.

1.6 Variables

Variables are used to store data. You can think of them like a container. The shape of the container is defined by the [Datatype](#) and then you can put any fitting value inside the container. Python variables are labels that point to an objects in memory that stores a value.

Summary

- **Definition:** Variables store references to objects in memory and are used to label and access these objects.
- **Usage:**
 - Create using `variable = value`; Python automatically determines the datatype.
 - Reassign variables to new or different types of values.
 - Use `del` to remove variables, freeing memory when unused.
- **Naming Rules:**
 - Must start with a letter or underscore and contain only letters, numbers, or underscores.
 - Avoid reserved keywords and descriptive naming is recommended.
 - Use `snake_case` or `camelCase` for readability.
- **Mutability:**

- Immutable objects (e.g., `int`, `str`) create new memory references on reassignment.
- Mutable objects (e.g., `list`, `dict`) allow in-place modification, affecting all references.
- **Scope:**
 - **Global:** Accessible program-wide.
 - **Local:** Restricted to within functions.
- **Type Hinting:** Use optional annotations (e.g., `x: int = 5`) for better clarity and debugging.

Memory Objects

🕒 partly exam relevant >

This topic might seem complicated and like an unnecessarily detailed description. However, working with memory references is extremely significant for low level languages, also in Python you can not ignore it. Because if you are not aware of it you, you can not properly understand Python, will regularly make hard to find mistakes and struggle. The details of how this works/behaves are also different between languages. Please try to understand the basics, even if the details seem hard.

In Python, variables do **not** directly store the values themselves. Instead, they store references to the objects in memory. A **reference** points to a location/address in computer memory where a object is stored.

The main characteristics of an object in Python are.

- A unique **identifier** (memory address) that distinguishes it from other objects. This is returned by the `id()` function.
- The **(data-)type** of the object that defines what kind of object it is (e.g., `int`, `str`, `list`). This is returned by the `type()` function.
- The **value** which is the actual data contained in the object (e.g., `42`, `"Hello"`, `[1, 2, 3]`). This is returned by default when just putting the name of the variable. Additionally, depending on the type of object, they can have Attributes and Methods.

```
1 x = 5          # creating a variable with the name x
2 print(id(x))   # print the object identifier/location in memory -> 140735731917896
3 print(type(x)) # print the type of the object -> <class 'int'>
4 print(x)       # print the value of the object -> 5
```

You can check if two variables point to the same object in memory using the `is` keyword.

Creating/Assigning

Use the assignment operator `=` to create and assign values to a variable. To do this you have to use the Syntax `variable_name = value`.

Variables in Python are not explicitly declared with a datatype. Instead, Python automatically picks a fitting type based on the assigned value at runtime. This also means/has the drawback that datatype errors/incompatibilities happen at runtime and can't easily be detected beforehand.

You can reassign a variable to a new value with the same syntax as creating one. You can change it to any value, even values of a different datatype.

```
1 number = 10      # name: number; value 10; type: Integer
2 message = "Hello" # name: message; value: "Hello", type: String
3 is_ready = True  # name: is_ready; value: True; type: Boolean
4
5 print(number)     # -> 10
6 number = "test"   # reassign variable number with value "test" of type String
7 print(number)     # -> test
```

Accessing

Use the variable name to access the value of a variable. Python will get the variable value from memory and put it place of the name. Tying to access a variable that does not exist will result in an error.

```
1 print(number) # -> 10
2 print(message) # -> Hello
```

Deleting

You can use the **keyword** `del` to delete a variable/the object reference it stores. If you are working with a **mutable** collection of type list or dictionary you can also use indexing and slicing to remove individual items.

```
1 x = 10          # creating x with value 10
2 del x          # deleting x
3
4 my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
5 del my_list[0]  # delete the first element
6 print(my_list)  # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]
7 del my_list[1:7:2] # delete every second item from the 2nd to the 8th
8 print(my_list)  # Output: [1, 3, 5, 7, 8, 9]
```

🕒 not exam relevant >

This is good to know, but not really exam relevant. To actually understand this it is good to first read [Objects Value vs. Reference](#).

If the reference count (how many times it is referenced for example by variables) of an object in memory gets down to zero it is no longer needed. Then Python's automatic garbage collector will free up the storage, so that it can be used for a different purpose again. So using `del` or exiting the [scope](#) of a variable will delete the reference and if there are no more references anywhere the object in memory will be deleted.

Naming Rules

There are a few rules that all variable names must follow.

1. **Start with a letter or an underscore:** A variable name must begin with a letter (a-z, A-Z) or an underscore `_`.
2. **Can contain letters, numbers, and underscores:** A variable name can contain any amount of letters, numbers and underscores, but no special characters or spaces.
3. **Cannot be a reserved keyword:** Avoid Python's built-in keywords like `if`, `for`, `class`, ...

Example of correct variable names: `x`, `_valid_name`, `variable123`, `_123_`, ...

Example of incorrect variable names:

```
1 2ndVariable = "Error" # Starts with a number
2 special!char = "Error" # Includes special character
3 space var = "Error"   # Includes space in variable name
4 class = "Error"       # Uses a reserved keyword
```

Naming Conventions

Naming conventions are not mandatory, but it is good to follow them. Good naming will create much easier to read, more consistent and better formatted code.

Variable names should be descriptive. So just by reading you should have a basic understanding of what value it will hold and what it is used for. So you should use descriptive words inside a variable name, without making it too long. Avoid shadowing built-in names (e.g., don't name your variable `float` or `list`). Avoid using single-character names except in specific contexts (e.g., `i` in loops). If there are multiple words in a variable name, you should consistently write it in `snake_case` or `camelCase`.

- in **snake case** words are separate by underscores `_` like this `this_variable_is_snake`
- in **camel case** words are separate by capitalizing them like this `thisVariableIsCamel`

Variable names are usually lowercase, except for potentially capitalizing all but the first word. Constants, which are variables that can not/never change their value, are not directly supported in Python. By convention, they are named all uppercase (`PI = 3.14159`).

Mutability Behaviour

🕒 partly exam relevant >

This topic might seem complicated and like an unnecessarily detailed description. However, working with memory references is extremely significant for low level languages, also in Python you can not ignore it. Because if you are not aware of it you, you can not properly understand Python, will regularly make hard to find mistakes and struggle. The details of how this works/behaves are also different between languages. Please try to understand the basics, even if the details seem hard.

Depending on whether a variable is storing a [mutable](#) or immutable object the behaviour can be significantly different. When assigning one variable to another `var2 = var1`, initially both of them will point to the exactly **same** location and therefore object in memory. So they do not only have the same value, they are actually the same thing. If you just want to copy the value while creating a new object you have to explicitly use methods like `.copy()`.

The interesting part starts when you change one of the variables. If the object is not [mutable](#), it is impossible to change its value in place. Instead a new object storing the changed value will be created and assigned to the changed variable. So at the end both of the variables will point at different objects with different values.

```
1 x = 1           # creating a variable x and assigning it the value 1
2 y = x          # creating a variable y and assigning to point to the same object as x
3 print(id(x), id(y)) # printing the memory address of x and y -> 140735731917608 140735731917608
4 print(x is y)   # checking if x and y point to the same object -> True
5 print(x, y)     # printing the values of x and y -> 1 1
6 y = y + 1       # creating a new object with the value of x + 1 and assigning it to y
7 print(id(x), id(y)) # printing the memory address of x and y -> 140735731917608 140735731917640
8 print(x is y)   # checking if x and y point to the same object -> False
9 print(x, y)     # printing the values of x and y -> 1 1
```

If you do a similar thing with an object that is [mutable](#), you can directly change its value in place. This means that the actual value of the object is changed. Since both of the variables are pointing to the same object, both of their values will change!

```
1 x = [1, 2, 3]   # creating a variable x and assigning it the value [1, 2, 3]
2 y = x           # creating a variable y and assigning to point to the same object as x
3 print(id(x), id(y)) # printing the memory address of x and y -> 1895382737088 1895382737088
4 print(x is y)   # checking if x and y point to the same object -> True
5 print(x, y)     # printing the values of x and y -> [1, 2, 3] [1, 2, 3]
6 y[0] = 10       # changing the value of the first element of the list y to 10
7                # since x and y point to the same object, the value of x will also change
```

```
8 print(id(x), id(y)) # printing the memory address of x and y -> 1895382737088 1895382737088
9 print(x is y)      # checking if x and y point to the same object -> True
10 print(x, y)       # printing the values of x and y -> [10, 2, 3] [10, 2, 3]
```

This might seem like a small difference and not intuitive at first, but is actually a very significant detail.

Scope

Variable scope means when/where variables are defined and can be accessed. In Python you are not directly accessing locations in memory, instead you are using context dependent variables.

Global Scope variables are outside of functions and can be accessed everywhere (after their creation).

Local Scope variables are created inside a function and because of that only exist inside that function. So after running the function is finished, all the references will be deleted again.

You can find detailed example and usage of the `global` and `nonlocal` [keywords](#) on the page about functions.

Type Hinting

🕒 not exam relevant >

This topic not exam relevant.

Even though Python automatically picks datatypes of variables based on their value, it supports type annotations for variables. This makes it easier to understand their intended type and can even be used by some debuggers. The syntax for variable type hinting is `varName: datatype = value`.

```
1 age: int = 25
2 name: str = "John"
3 height: float = 5.9
```

This simple type hinting works with most datatypes. With many collection datatypes you can also define the inside elements datatypes.

Name	Type Hint	Syntax Example
List	list[<type>]	numbers: list[int] = [1, 2, 3]
Tuple	tuple[<type1>, <type2>, ...]	coords: tuple[int, float] = (1, 0.5)
Set	set[<type>]	unique_nums: set[float] = {0.1, 2.5, 0.3}
Dictionary	dict[<key_type>, <value_type>]	person: dict[str, int] = {"age": 30}
Multidimensional List	list[list[<type>]]	matrix: list[list[int]] = [[1, 2], [3, 4]]
List of Tuples (String and Range)	list[tuple[str, range]]	data: list[tuple[str, range]] = [("A", range(3)), ("B", range(1,99,3))]

Questions

- ☐ Explain variables and their usage in your own words.
- ☐ Explain Objects in python.
- ☐ What is the meaning/difference of variable, value, reference, object, and (data-)type in python
- ☐ Show and explain the creating, accessing, updating and deleting of variables.
- ☐ Do you have to tell Python which datatype a variable has?
- ☐ What are the rules for naming variables in Python?
- ☐ How can you make good variable names and why is this important?
- ☐ How does Python handle variable reassignment with mutable vs. immutable objects?
- ☐ How do you check whether two variables point to the same object in memory?
- ☐ Explain variable scope in your own words.
- ☐ What happens if you try to access a variable that hasn't been defined?
- ☐ Explain type hinting in your own words.

1.7 Operators

Operators in Python are special symbols or keywords used to perform operations on variables and values. An Operator will usually take one or more variables/values called operands, perform an operation and return the result. Python supports a variety of operators categorized by their functionality.

Summary

- `=` : Assigns values to variables. Can be combined with other operations, e.g., `x += 1`, `x /= 3`.
- Arithmetic Operators**
 - Standard operators like `+`, `-`, `*`, `/`, `//`, `%`, `**` for mathematical operations.
 - Floor Division `//` Rounds down to nearest whole number.
 - Modulo `%` Returns remainder after division.
- Relational Operators are used for comparisons, returning boolean results: `==`, `!=`, `>`, `<`, `>=`, `<=`.
- Logical Operators** chain conditions together
 - `and` : [Truthy](#) if both conditions are Truthy.

- or : [Truthy](#) if at least one condition is Truthy.
- not : Inverts the boolean value.
- Operators are evaluated based on precedence, e.g. `()` , `**` , `*` / , `+` - , comparison, logical, assignment

Assignment Operator

The assignment operator `=` is used to [assign](#) a value to a [variable](#) .

```
1 x = 5 # assign the value 5 to the variable x
```

🕒 [partly exam relevant](#) >

The following 3 subchapters show advanced usage. It is good to know about this, but not strictly exam relevant.

Augmented Assignment

You can combine the assignment operator with an arithmetic or bitwise. This will apply the operation to the current value of a variable and then assign the resulting value to the variable again. To do this you have to put the other operator between the variable name and the assignment.

```
1 x = 1      # assign the value 1 to the variable x
2 x += 1     # same as 'x=x+1', add one to the value of x, result: 2
3 x *= 3     # same as 'x=x*3', multiply the value of x with 3, result: 6
4 x -= 2 * 4 # same as 'x=x-2*4', result -2, because of precedence
5 x **= 2    # same as 'x=x**2', result 4
6 x <<= 1    # same as 'x=x<<2', result 8
7 x //= 3    # same as 'x=x//3', result 2
```

Multiple Value Assignment

Python allows you to assignment multiple values at once using the syntax `var1, var2, var3 = val1, val2, val3` . You can also use this with [iterable](#) values like this `var1, var2, var3 = (val1, val2, val3)` . If there are more values than variables, you have to tell python which variable should take a list of multiple values by marking it with an asterisk `*` like this `var1, *others = (val1, val2, val3, val4)` .

```
1 a, b, c = 1, True, "test"      # multiple value assignment
2 print(a, b, c)                 # -> 1 True test
3
4 a, b = b, a                     # swap the values of two variables
5 print(a, b)                     # -> 2 1
6
7 a, *b = (1, 2, 3, 4, 5)         # a takes the first, b takes a list of all others
8 print(a)                       # -> 1
9 print(b)                       # -> [2, 3, 4, 5]
10
11 *a, b, c = 1, 2                # c takes the last, b the second last, a takes all extra, which are in this case no elements at all
12 print(a)                      # -> []
13 print(b)                      # -> 1
14 print(c)                      # -> 2
15
16 a, *b, c = "This is some text" # a takes the first, c takes the last, b takes all in the middle as a list
17 print(a)                      # -> T
18 print(b)                      # -> ['h', 'i', 's', ' ', 'i', 's', ' ', 's', 'o', 'm', 'e', ' ', 't', 'e', 'x']
19 print(c)                      # -> t
```

Walrus Operator

🕒 [not exam relevant](#) >

Not exam relevant, rarely used, but sometimes really helpful.

The walrus operator `:=` combines assignment and returning the value for further use. This can be helpful to reduce code duplication, especially with conditions or loops. The walrus operator can also be combined with augmented assignment.

```
1 # Without Walrus Operator
2 value = input("Enter 'exit' to stop the loop: ")
3 while value != "exit":
4     print("Inside while loop with input:", value)
5     value = input("Enter 'exit' to stop the loop: ")
6 print("loop finished")
7
8 # With Walrus Operator
9 while (value := input("Enter 'exit' to stop the loop: ")) != "exit":
10 # this will take input, assing it to the variable value, and also return it to also be used in for value != "exit"
11     print("Inside while loop with input:", value)
12 print("loop finished")
```

Arithmetic Operators

Arithmetic operators are basically the operators you already know from mathematics. With numbers types like `int` , `float` or `complex` they will work the same as you expect from mathematics.

Name	Operator	Syntax	Example
Addition or Concatenation	+	a + b	3 + 2 = 5 , "Hello" + "World" = "HelloWorld"
Subtraction	-	a - b	5.5 - 2 = 3.5
Multiplication or Repeating	*	a * b	-1.2 * 3 = -3.6 , 4 * "Hi" = "HiHiHiHi"
Division	/	a / b	7 / 2 = 3.5
Floor Division	//	a // b	7 // 2 = 3
Modulus/Remainder	%	a % b	7 % 2 = 1
Exponentiation (Power)	**	a ** b	3 ** 2 = 9

Exponentiation also works with floats, and fractional exponents can be used to calculate roots.

Division Operators

Python actually has three different operators related to division. The true division `/` will give you an accurate division result as a float number. The floor division `//` will return the result of doing the division and then **rounding down** to the next lowest whole number. The modulo `%` will return the **remainder** when doing a whole number division.

```
1 print(f"The result of dviding 13 by 5 is {13/5}") # The result of dviding 13 by 5 is 2.6
2 print(f"This means that 5 fully fits into 12 {13//5} times") # This means that 5 fully fits into 12 2 times
3 print(f"When fitting 5 into 13, there will be a remainder of {13%5}") # When fitting 5 into 13, there will be a remainder of 3
```

 [less exam relevant >](#)

Less exam relevant, but still good/helpful to know.

So mathematically speaking: `a % b = a - b*(a//b)` . One common use case for the Modulus is to check divisibility by checking if the remainder is 0. It can also be used to separate parts of a number. You can also think of the modulo in terms of a circle. For example when working with mode 360° $0^\circ = 360^\circ = 720^\circ \dots$

```
1 print(0%2, 1%2, 2%2, 3%2, 4%2, 5%2, 6%2, 7%2) # -> 0 1 0 1 0 1 0 1
2 print(12%7, 73%7, 49%7, 111%7, 14%7, 71414%7) # -> 5 3 0 6 0 0
3 print(3%10, 16%10, 211%10, 9873%10, 1357%100) # -> 3 6 1 3 57
4 print((1357//100)%10, (123456789//1000)%1000) # -> 3 456
5 print(10%360, 370%360, 730%360, 3610%360) # -> 10 10 10 10
```

Result Datatype

The [datatype](#) of the result of an operation only depends on the datatypes or the two operands.

Addition `+` , Subtraction `-` , Multiplication `*` , Floor Division `//` , Modulus `%` and Exponentiation `**` will return an integer if both of the operands are [integers](#). If one of them is [complex](#) the result is complex. Otherwise, if one of them is a [float](#), the result will be float.

The true Division `/` will always return a [float](#). If one of the operands is [complex](#), it will return complex.

```
1 a = 3
2 b = 1.5
3 c = 2 - 1j
4 print(a + b) # -> 4.5
5 print(b - c) # -> (-0.5+1j)
6 print(b * c) # -> (3-1.5j)
7 print(a / a) # -> 1.0
8 print(2 * b // 1) # -> 3.0
```

When using operators with collections the result datatype is the same as the datatype of the collection you use.

Collection Usage

The operators `+` and `*` can also be used with [sequence datatypes](#). The `+` operator is used to concatenate/combine two strings, lists or tuples of the **same** datatype.

```
1 x = "Hello"
2 print(x + " " + "World") # -> Hello World
3 print([1, 2] + [3, 4]) # -> [1, 2, 3, 4]
4 print((1, 2) + (3, (True, "Test"))) # -> (1, 2, 3, (True, 'Test'))
5 print(x + [1, 2]) # ERROR because they have different datatypes
```

 [not exam relevant >](#)

Not exam relevant, rarely used, but sometimes really helpful.

The `*` operator is used to repeat a string, list or tuple multiple times. For that you have to give the number of time you want to repeat as a positive integer and the sequence you want to repeat. The result datatype is the same as you start with.

```
1 print(3 * "Hello") # -> HelloHelloHello
2 print([0] * 10)    # -> [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
3 print((1,2,3) * 2) # -> (1, 2, 3, 1, 2, 3)
```

Relational/Comparison Operators

Relational Operators always compare two values and return a [boolean](#).

Name	Operator	Example	Explanation	Opposite
Equal	<code>==</code>	<code>a == b</code>	True if <code>a</code> and <code>b</code> have the same value.	<code>!=</code>
Not Equal	<code>!=</code>	<code>a != b</code>	True if <code>a</code> and <code>b</code> do not have the same value.	<code>==</code>
Greater Than	<code>></code>	<code>a > b</code>	True if <code>a</code> is greater than <code>b</code> .	<code><=</code>
Less Than	<code><</code>	<code>a < b</code>	True if <code>a</code> is less than <code>b</code> .	<code>>=</code>
Greater or Equal	<code>>=</code>	<code>a >= b</code>	True if <code>a</code> is greater than or equal to <code>b</code> .	<code><</code>
Less or Equal	<code><=</code>	<code>a <= b</code>	True if <code>a</code> is less than or equal to <code>b</code> .	<code>></code>

If you look at the logic, you can see that some of them are exact opposites (`not` operator).

(In-)Equality

Equality `==` and Inequality `!=` are exact opposites and work with all datatypes. The Equal operator checks if two values are the same. Comparing two numbers will ignore datatype differences and implicitly converted. When comparing a boolean to a number `True == 1` and `False == 0`.

```
1 print(1 == 2)          # -> False
2 print(1 == 1.0)        # -> True
3 print(1 == 1.0000001)  # -> False
4 print(1.5 != 2-1j)     # -> True
5 print(0.3 == 0.3+0j)   # -> False
6 print(True == 1)       # -> True
7 print(True != False)   # -> True
8 print([1, 2] == (1, 2)) # -> False
9 print("test" != "Test") # -> True
```

Greater/Less Than

Check if one value is greater than `>` or smaller than `<` a different one. Primarily used to compare integer or float numbers, which are compatible with each other. If an `=` is included (`>=` or `<=`), having to equal values will also return `True`.

```
1 print(1 > 1)          # -> False
2 print(1 >= 1)         # -> True
3 print(1 < 1.2)        # -> True
4 print(-1 > -2.5)      # -> True
5 print(1 >= 1.0)       # -> True
6 print(1 <= 1.0)       # -> True
```

🕒 not exam relevant >

Not exam relevant, rarely used, but sometimes really helpful.

These operators can also be used with the [sequence datatypes](#) string, list and tuple. Both operands must be of the same datatype. This will check the operation for all pairs of elements one by one. So as soon as there is a pair of elements that do not have the same value it will result in a total result of either `True` or `False`. If they have different lengths but the element pairs are all equal, the shorter sequence is considered smaller.

```
1 print([1] < [2])          # because 1<2 -> True
2 print([1, 3] < [1, 2])    # because 3<2 -> False
3 print([1, 0.5, 9] < [1, 1, 0]) # because 0.5<1 -> True
4 print([1, 2, 3] < [1, 2, 3]) # because equal -> False
5 print([1, 2, 3] <= [1, 2, 3]) # because equal -> True
6 print([1, 2, 3] < [1, 2, 3, -99]) # because list 1 is shorter and the common elements are same-> True
7 print("a" < "b")          # -> True
8 print("computer" < "comparison") # because "u"<"a" -> False
```

Chained Comparison

🕒 not exam relevant >

Not exam relevant, rarely used, but sometimes really helpful.

Python allows you to chain comparisons together. This will behave the same way as having an `and` between the individual operations. For example `a < b == c >= d` is the same as `a < b` and `b == c` and `c >= d`. The easiest use case is to check if a number is between two others.

```
1 x = 4
2 print(1 < x < 10)      # -> True
3 print(4 < x < 10)      # -> False
4 print(0 < x < 10 < x**2) # -> True
```

Logical Operators

Logical operators are used to combine multiple comparisons and Booleans to more complex logical statements.

Operator	Example	Explanation
and	a and b	True if both a and b are True.
or	a or b	True if at least one (a or b) is True.
not	not a	True if a is False (logical negation).

```
1 print(False and False)      # -> False
2 print(False and True)       # -> False
3 print(True and True)        # -> True
4 print(False or False)       # -> False
5 print(False or True)        # -> True
6 print(True or True)         # -> True
7 print(not False)            # -> True
8 print(not True)             # -> False
9 print(2 == 1 or 2 == 2 or 2 == 3) # -> True
```

Not Boolean Operands

🕒 not exam relevant >

Not exam relevant, rarely used, but sometimes really helpful.

As long as the operands are boolean, the result will also be boolean. However in Python logical operators also work with operands that are not boolean. They will keep their general behavior and `not` will always return a boolean. The `and` operator stops as soon as it encounters a [Falsy](#) value and returns it. If all values are [Truthy](#), it returns the last operand. The `or` operator stops as soon as it encounters a Truthy value and returns it. If all values are Falsy, it returns the last operand.

```
1 print(0.0 and 1)            # -> 0.0
2 print(0 or 0.0 or "" or "Default") # -> Default
3 print(1 and 2 and 3)        # -> 3
4 print((5 or 0) and (" or "World")) # -> World
```

Collection Operators

[Indexing](#) and [Slicing](#) are explained in detail on the datatypes page.

Membership Checking

The operator `in` is used to check if a value/element exists in a collection. It always returns a [boolean](#) and uses the syntax `value in collection`. So basically it checks if any of the elements in the collection is [equal](#) to the value.

```
1 print(1 in [0, 1, 2])      # -> True
2 print(3 in (0, 1, 2))     # -> False
3 print("good" in {"bad", "ok", "good"}) # -> True
4 print(0.0+0j in [0, 1, 2, 3]) # -> True
5 print((1, 2) in (1, 2, 3, 4)) # False
6 print(3 in [(1,2), (3,4), (5,6)]) # -> False
7 print([3,4] in [(1,2), (3,4), (5,6)]) # -> False
8 print((3,4) in [(1,2), (3,4), (5,6)]) # -> True
```

When this operator is used with a [dictionary](#), it will check if one of the **keys** is equal to the value. If you want to check if a value exists in a dictionary, you have to use the built-in method `.values()`.

```
1 data = {"name": "Alice", "age": 25, "city": "New York", 1: "test"}
2 print("name" in data)      # -> True
3 print(1 in data)           # -> True
4 print("Alice" in data)     # -> False
5 print("Alice" in data.values()) # -> True
```

When using this operator with a [string](#), it checks if the value is a substring of the whole string.

```

1 print("d" in "andreas")           # -> True
2 print("word" in "check for whole word") # -> True
3 print("ork" in "This works also")     # -> True
4 print("IT" in "capitalization matters") # -> False
5 print("IT" in "CAPITALIZATION MATTERS") # -> True

```

You can also check if a value is not in a collection using the syntax `not (value in collection)` or `value not in collection`.

🔗 Additional not exam relevant topics for later >

Set Operators

Dictionary Operators

Collection Unpacking

Bitwise Operators

Precedence

Operator precedence determines the order in which operators are evaluated. If two have the same precedence python works from left to right. This is similar to mathematical operations where you also have to keep the correct PEMDAS (Parentheses, Exponents, Multiplication and Division, Addition and Subtraction) order

Operator	Description
()	Parentheses (used to group expressions)
**	Exponentiation
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise left shift, Bitwise right shift
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, <, >=, <=	Comparison operators
is, is not, in, not in	Identity and membership tests
not	Logical NOT
and	Logical AND
or	Logical OR
=, +=, -=, etc.	Assignment and augmented assignment operators
lambda	Lambda function declaration

Operator	Description
()	Parentheses (used to group expressions)
**	Exponentiation
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise left shift, Bitwise right shift
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, <, >=, <=	Comparison operators
is, is not, in, not in	Identity and membership tests
not	Logical NOT
and	Logical AND
or	Logical OR
if ... else	Ternary Operator
=, +=, -=, etc.	Assignment and augmented assignment operators
lambda	Lambda function declaration

Questions

☐ Explain assignment in you own words.

- ☐ Explain augmented assignment in you own words.
- ☐ Explain multiple value assignment in you own words.
- ☐ Explain the walrus operator in you own words.
- ☐ Explain arithmetic operators in you own words.
 - ☐ Explain the difference between `/` , `//` and `%` in detail.
 - ☐ Explain what datatype the result of an operation will be.
 - ☐ Explain the arithmetic operators with sequence datatypes in you own words.
 - ☐ Compare using contaminating vs. formatted strings to put together a multiple part string with different variables including a practical example.
- ☐ Explain comparison operators in you own words.
 - ☐ Explain `==` and `!=` in detail. What is the difference between `==` and `is` .
 - ☐ Explain greaten than and less then in detail.
 - ☐ Explain chained comparisons in you own words.
- ☐ Explain logical operators in you own words.
 - ☐ Explain logical operators with operands that are not boolean in detail
- ☐ The membership operator `in` in you own words.
 - ☐ What is special when using it with a `string` ?
- ☐ Explain operator precedence in detail in you own words.
 - ☐ Show operator precedence by creating and explaining practical examples.

1.8 Keywords

In Python, **keywords** are reserved words that have a special predefined meaning and cannot be used as identifiers (e.g., variable names, function names). Keywords are the building blocks of Python syntax and structure.

Most keywords are explained an a different page, so the "Context" column will link to the details.

🕒 partly exam relevant >

The keywords `match` , `case` , `default` , `pass` , `lambda` , `yield` , `nonlocal` , `async` , and `await` are not exam relevant. Some of the others are also only partly or for some departments exam relevant.

Context	Keyword	Description
Datatype	<code>False</code>	Boolean value representing False.
Datatype	<code>True</code>	Boolean value representing True.
Datatype	<code>None</code>	Represents the absence of a value or a null value.
Operator	<code>and</code>	Logical AND operator. True if all operands are True.
Operator	<code>or</code>	Logical OR operator. True if at least one operand is True.
Operator	<code>not</code>	Logical NOT operator. Negates a boolean value.
Operator	<code>in</code>	Checks if an element exists in a sequence or collection. Also used to iterate a variable in a for loop.
Variable	<code>is</code>	Check if two objects have the same identity (=location in memory).
Variable	<code>del</code>	Delete a object, variable, or an items from a collection.
Variable	<code>global</code>	Declares a variable as global, accessible outside the current scope.
Variable	<code>nonlocal</code>	Refers to a variable in the nearest enclosing scope (not global).
Flow Control	<code>if</code>	Defines a conditional statement.
Flow Control	<code>elif</code>	Conditional branching; short for "else if".
Flow Control	<code>else</code>	Defines the block of code to execute if the <code>if</code> condition is False. Also has a special use-case in the context of while , for and except.
Flow Control	<code>match</code>	Used for pattern matching. Executes blocks of code based on the structure or value of an expression.
Flow Control	<code>case</code>	Defines a block within a <code>match</code> statement, representing a specific case to match a value or pattern.
Flow Control	<code>default</code>	Used in a <code>match</code> statement to handle no <code>case</code> is matching.
Flow Control	<code>while</code>	Defines a loop that continues as long as the condition is True.
Flow Control	<code>for</code>	Defines a loop that iterates over a sequence.
Flow Control	<code>break</code>	Terminates/jumps after the nearest enclosing loop.
Flow Control	<code>continue</code>	Skips the remaining code in a loop iteration and moves to the next iteration.
Flow Control	<code>pass</code>	Placeholder statement that does nothing.
Function	<code>def</code>	Defines a new function or method.
Function	<code>lambda</code>	Creates an anonymous function (inline function).
Function	<code>return</code>	Exits a function and optionally passes back a value.
Function	<code>yield</code>	Pauses a function, returning a value, and allows it to be resumed later (used in generators).
OOP	<code>class</code>	Defines a new function or class.
Exception	<code>try</code>	Defines a block of code to test for exceptions.
Exception	<code>except</code>	Defines how to handle exceptions in a <code>try</code> block.
Exception	<code>finally</code>	Defines cleanup code that always runs after <code>try</code> or <code>except</code> blocks.

Context	Keyword	Description
Exception	raise	Used to raise exceptions manually.
Exception	assert	Debugging tool to test if a condition is not True, it will throw an exception.
Exception	with	Context Manager to properly enter and exit resources like files.
Import	import	Imports a module or specific components from a module.
Import	from	Specifies the module from which to import.
Import / Exception	as	Used for creating aliases/alternative names.
Asynchronous	async	Used to define asynchronous functions (coroutines).
Asynchronous	await	Used to pause execution in asynchronous functions until a result is available.

Questions

☐ What is a keyword? Explain in your own words.